

PROBLEM STATEMENT 13

Air-Gapped Predictive Copilot for Secure MPLS Operations

Description

Modern enterprise and government networks increasingly rely on SD-WAN deployments running over MPLS underlays to deliver resilient, application-aware connectivity across distributed branches, datacenters, and cloud environments.

As these networks grow in complexity, operational visibility and response speed become critical to maintaining service quality and security posture.

Conventional NOC tooling remains predominantly reactive — faults are detected only after user-visible service degradation has occurred.

Two compounding challenges define this operational gap:

- *Reactive Detection*

Threshold-based alerts fire only after performance thresholds are breached, providing no time for pre-emptive intervention.

- *Air-Gap Constraints*

Regulated and government environments prohibit cloud-connected AI inference tools, leaving operators without intelligent guidance in the most security-sensitive deployments.

This problem statement calls for an autonomous, air-gapped offline AI NOC Copilot that predicts network failures before operational impact, explains reasoning in natural language, and operates entirely within an air-gapped network.

Objectives

Objective 1 — Simulated SD-WAN/MPLS Environment

Construct a reproducible, multi-site simulated network topology representative of real-world enterprise deployments, including:

- Branch, hub, and datacenter sites with CE/PE/P device roles
- MPLS forwarding plane, VPN segmentation, and traffic engineering constructs
- SD-WAN IPSec overlay tunnels, dynamic routing (BGP/OSPF), and QoS policies
- Realistic application traffic flows and configurable fault injection capabilities

Objective 2 — Predictive Fault Analytics Engine

Develop machine learning and statistical models that detect precursor conditions rather than threshold breaches:

- Time-series forecasting for congestion buildup, interface utilization saturation, and latency drift
 - Routing instability detection — BGP/OSPF convergence stress, route flapping precursors, path asymmetry
 - Tunnel health degradation scoring — packet loss progression, jitter trends, rekey anomalies
 - Time-to-impact estimation providing actionable lead times before service breach
-

Objective 3 — Offline LLM NOC Copilot

Deploy a fully self-hosted, air-gapped LLM to provide natural-language decision support:

- Local model packaging — quantized LLM bundled within the air-gapped environment
 - Retrieval-Augmented Generation (RAG) over internal artifacts only — topology maps, runbooks, past incidents
 - Structured copilot responses including predicted issue, confidence score, root-cause hypothesis, affected scope, and recommended actions
 - Natural-language query interface for NOC operators
-

Objective 4 — Integrated NOC Workflow Automation

Minimize manual troubleshooting effort by automating key NOC workflows:

- Continuous topology awareness and dynamic graph-based event correlation
 - Confidence-scored alert prioritization to reduce alert fatigue
 - Automated playbook suggestion and action sequencing
 - Operator-ready incident summaries with estimated impact and urgency classification
-

Expected Outcomes

The platform must enable NOC operators to answer three operational questions in real time:

Q1: What is likely to fail next — and when?

Q2: Why is risk assessed as elevated — which signals contributed?

Q3: What corrective action should be taken before SLA or security impact occurs?

Success is defined not by whether the system can detect a failure that has already occurred, but by whether it can forecast degradation with sufficient lead time for the NOC to intervene preventively, and whether the LLM copilot can communicate that forecast in operator-ready language without any dependency on external networks or cloud APIs.

Dataset Required

All data is generated within the simulated environment and must remain within the air-gapped boundary:

- SNMP interface utilisation, latency, jitter, and error counters
 - Syslog and routing protocol events (BGP/OSPF adjacency changes, route advertisements)
 - NetFlow/IPFIX flow records and tunnel statistics
 - Streaming telemetry from SD-WAN controllers
 - Injected fault and adversarial scenario ground-truth labels for model training and validation
-

Suggested Tools / Technologies (but not limited to)

Network Simulation

- EVE-NG
- GNS3
- Containerlab

Telemetry Pipeline

- Telegraf
- Prometheus
- Elasticsearch
- Kafka

Predictive Models

- LSTM
- Prophet
- Graph-based anomaly detection

- Ensemble classifiers

Offline LLM

- Mistral 7B
- LLaMA 3 8B
- Phi-3 (quantized for on-premises deployment)

RAG / Vector Database

- Local deployment (no cloud dependency)

Additional Components

- Traffic generation tooling
- Fault injection tooling

Expected Solution / Steps to be Followed to Achieve the Objectives

Phase 1 — Network Simulation

Build the simulated SD-WAN over MPLS topology.

Configure multi-site topology with branch, hub, and datacenter nodes; establish MPLS forwarding, VPN segmentation, dynamic routing protocols, and overlay tunnels.

Deploy traffic generation tools and implement fault injection capabilities for scenario validation.

Phase 2 — Telemetry Pipeline

Deploy a local telemetry collection stack to ingest and normalise signals from all simulated devices.

Align interface utilisation, latency, jitter, BGP/OSPF events, tunnel statistics, syslog events, controller changes, and flow records into a time-series dataset.

All data remains within the air-gapped boundary.

Phase 3 — Predictive Modelling

Train and validate predictive models against historical telemetry using injected fault scenarios as ground truth.

Evaluate model candidates on:

- Precision
- Recall
- False-positive rate
- Prediction lead time

Select or ensemble the best-performing combination for production inference.

Phase 4 — Offline LLM Deployment

Select and quantize a compact open-source LLM for on-premises deployment.

Package the model with all runtime dependencies into a portable bundle with all outbound network access disabled.

Implement a RAG pipeline connecting the LLM to a local vector database populated with:

- Topology metadata
 - Alert context
 - Runbooks
 - Past incident records
-

Phase 5 — Copilot Integration & Decision Support

Wire predictive model outputs and network telemetry into the LLM context window via the RAG pipeline.

Configure the copilot to produce structured responses for every alert, including:

- Predicted issue type
 - Confidence score
 - Probable root cause
 - Affected sites and services
 - Estimated time-to-impact
-

Phase 6 — Scenario Validation

Inject a set of realistic fault and adversarial scenarios and measure platform response.

For each scenario, record:

- Prediction lead time
- Copilot explanation quality

- Accuracy of recommended remediation

Test Scenarios

1. Progressive congestion buildup on a hub-spoke link
2. BGP route flap with downstream path reroute cascade
3. Intermittent MPLS underlay failure with tunnel degradation
4. Controller misconfiguration leading to policy drift

Evaluation Parameters

Evaluation Dimension Weight		Key Criteria
Technical Merit	35%	Prediction accuracy and lead time — how early and how accurately does the platform forecast faults and anomalies before impact?
Copilot Effectiveness	35%	Copilot quality — are explanations correct, operator-relevant, and grounded in local retrieval without hallucination?
Security & Offline Compliance	20%	Air-gap integrity — verifiably zero outbound dependency during runtime; offline security controls implemented.
Documentation Quality	10%	Clarity of documentation, architecture, and design rationale.